

## Solution Architecture

|                      |                                                              |
|----------------------|--------------------------------------------------------------|
| <b>Date</b>          | 2 October 2026                                               |
| <b>Team ID</b>       | SWTID-2026-6582                                              |
| <b>Project Name</b>  | PocketSmart AI: Your Smart Budget & Recommendation Assistant |
| <b>Maximum Marks</b> | 4 Marks                                                      |

## Solution Architecture

The architecture separates presentation, routing, AI generation, deterministic validation, persistence and external navigation. This prevents the AI model from being the sole source of financial truth.

| Layer                 | Component / Technology                           | Responsibility                                                                     |
|-----------------------|--------------------------------------------------|------------------------------------------------------------------------------------|
| Client Layer          | Browser, Jinja2 templates, HTML/CSS/JavaScript   | Collects inputs and displays results.                                              |
| API Layer             | FastAPI routers + Uvicorn                        | Handles routes, forms, auth dependencies and responses.                            |
| Orchestration Layer   | recommendation_service.py                        | Coordinates Gemini, revision loop, budget engine, search links and persistence.    |
| AI Layer              | gemini_service.py + Google Gemini 2.5 Flash Lite | Generates structured recommendations and performs multimodal outfit analysis.      |
| Validation Layer      | budget_service.py + Pydantic                     | Calculates exact totals and validates schema/budget constraints.                   |
| Persistence Layer     | SQLAlchemy + SQLite                              | Stores users and authenticated recommendation history.                             |
| External Search Layer | product_provider.py                              | Builds search URLs for marketplaces/service platforms; does not fetch live prices. |

## Architecture & Data Flow

USER → Browser/Jinja2 UI → FastAPI Router → Pydantic Validation → Recommendation Service → Gemini Service → Google Gemini API → Structured Response → Python Budget Engine → Product Search Link Generator → SQLite (if authenticated) → Jinja2 Result → USER

## Jewelry Multimodal Path

Outfit Image → Image Validation (Pillow, type/size/integrity checks) → PIL Image + text prompt → Gemini multimodal request → outfit color/style/formality analysis → jewelry recommendations → Python budget validation → result page.